Experiments and Results

4.1 Implementation

This section describes how the Graph-RAG pipeline of Chapter 3 is realised as a working system: the data (Section 4.1.1), the software architecture (4.1.2), retrieval and ranking (4.1.3), the use of the large language model (LLM) (4.1.4), and the validation of its output (4.1.5).

4.1.1 Data

The knowledge graph is built from the Decagon dataset [1], which the Stanford SNAP BioSNAP collection distributes as four CSV files; their sizes are given in Section 1.3.2. The fourth file, which lists single-drug side effects, is optional and serves only as a ranking feature (Section 4.1.3), never for retrieval.

Two structural properties of the graph shaped the implementation. First, both the drug–target and the protein–protein subgraphs are heavily right-skewed (Table 4.1): half of all drugs bind at most 11 proteins, yet one binds 2,840, and half of all proteins have at most 33 interaction partners, yet the busiest has 2,519. Letting such hub proteins act as bridges would connect almost any two drugs through an uninformative path. Setting `hub_degree_threshold` to 500 (Table 4.3) removes only the extreme tail, 220 of 19,081 proteins (1.15%), close to the 99th percentile.

Table 4.1: Degree-distribution statistics of the drug–target and protein–protein graphs

Degree	**Min**	**Median**	**Mean**	**P90**	**P99**	**Max**
Targets per drug	1	11	65.8	170	206	2,840
Drugs per target protein	1	1	5.1	5	78	94
PPI neighbours per protein	1	33	75.0	201	525	2,519

Second, the 645 drugs with documented pairwise history in `bio-decagon-combo.csv` are not the same population as the 284 drugs in `bio-decagon-targets.csv`: only 284 of 645 (44%) have a known target set. For the other 361, Tier 1 evidence (Table 3.2) is available whenever the pair was tested, but Tier 2 and Tier 3 need a protein-level path and are unreachable.

Third, the documented history is heavy-tailed at two levels (Table 4.2). A drug pair with any documented effect has a median of 53 and up to 540, which is why effects are ranked (Eq. (4.1)) rather than listed; and an effect code is documented for a median of 1,596 pairs but for as few as one, which is why the inverse-frequency term of Eq. (4.2) matters. Of the 1,317 effect codes in the raw file, 963 occur in at least 500 pairs, matching within one the 964 effects that Zitnik et al.

[1] evaluate after the same cut-off; the remaining ~350 receive a larger $I(e)$ rather than being dropped.

Table 4.2: Side-effect frequency statistics in bio-decagon-combo.csv

Quantity	Min	Median	Mean	P90	P99	Max
Effects per documented pair	1	53	73.3	165	305	540
Pairs documenting an effect	1	1,596	3,530.3	9,530	21,396	28,568

At run time, drug–target and protein–protein edges are loaded once into in-memory adjacency maps for constant-time neighbour look-ups. The 4,649,441-row combination file is materialised once into a local SQLite table `side_effects` keyed by (drug pair, effect code) with an index on the second drug, plus two derived tables used by the ranking formula: `effect_counts` (n_e, pairs documenting each effect) and `stats` (N, distinct documented pairs). A SHA-256 signature of each source file and a schema version are stored with it, so the cache is rebuilt only when a source file or the schema changes.

Free text typed by a user, including Vietnamese brand names, typos, or several drugs in one sentence, is resolved to a Decagon STITCH identifier by a rule-based resolver: (i) a STITCH-ID pattern, (ii) an exact match on normalised generic names, synonyms, or Vietnamese brand aliases, then (iii) fuzzy string matching (RapidFuzz), accepted automatically above a score of 90 and offered as a suggestion above 75. A curated stop-word list removes conversational filler first, so that *"em đang uống panadol với ibuprofen"* resolves to the two intended drugs. Generic names, brand aliases, gene symbols, Vietnamese effect names and the EMA seriousness lists come from resource tables built offline.

4.1.2 Software Architecture

The system is written in Python. `app.py` is a Streamlit front-end that calls the single entry point `PolypharmacySystem.predict()` of `pipeline.py`. The package `graphrag/` holds `config.py` (settings and ranking weights), `dataset.py` (loading, SQLite cache, subgraph retrieval), `names.py` and `resolver.py` (name resolution), `ranking.py` (effect scoring), `llm.py` (prompt, API call, validation), `presenter.py` (assembling results) and `visualize.py` (interaction graph). Paths, the LLM endpoint and model, and the parameters of Table 4.3 are read from a `.env` file, so they can change without editing code. Dependencies are kept small: `httpx` for the LLM call, `pydantic` for output validation, `rapidfuzz` for name resolution and `pyvis` for the interaction graph.

4.1.3 Subgraph Retrieval, Pruning, and Ranking

Retrieval follows the six steps of Table 3.1 within the bounds of Table 4.3. A protein is pruned as a hub once its PPI degree exceeds `hub_degree_threshold`: it is never used as a bridge intermediate, although a direct target of a queried drug stays an endpoint. Bridges are searched only for pairs with no recorded effect at all, are bounded per pair, and are visited in a fixed order, so results are reproducible and truncation is flagged. The evidence sent to the LLM is capped at `max_evidence` and shared across drug pairs round-robin, so one pair cannot use the whole budget. Documented (Tier 1 and 2) evidence is built only for the effects that will be shown, and Tier 2 and Tier 3 each have a reserved quarter of the budget, whatever a tier leaves unused going to the others. Without this reservation Tier 1 alone filled the budget in larger regimens: in 40 six-drug regimens built around a common target, no Tier 2 or Tier 3 evidence

reached the LLM; with it, every one of them carries both. Paths through pruned hubs are removed before the budget is spent.

Table 4.3: Retrieval and pruning bounds (default values)

Parameter	Default	Role
`hub_degree_threshold`	500	PPI degree above which a protein is pruned
`max_evidence`	400	Cap on evidence items serialised to the LLM
`tier3_one_hop_per_pair`	8	One-hop bridges sampled per undocumented pair
`tier3_two_hop_per_pair`	4	Two-hop bridges sampled per undocumented pair
`tier3_candidates_per_pair`	20	Candidate labels offered for a Tier 3 pair
`max_serious_effects`	10	Serious effects surfaced per analysis
`max_other_effects`	5	Other effects surfaced per analysis
`hub_degree_percentile`	0 (off)	Hub threshold as a PPI degree percentile
`convergence_amplification`	off	Count one-hop target convergence as Tier 2

Because a pair can list dozens to hundreds of effects, every documented effect e in the regimen receives a transparent additive score

$$\text{score}(e) = w_r R(e) + w_i I(e) + w_m M(e) + w_s S(e) + w_c S(e) C(e) + w_a A(e), \qquad (4.1)$$

where $R(e)$ is the number of regimen pairs documenting e; $M(e)$, $S(e)$ and $C(e)$ are binary indicators for "absent from both drugs' individual side-effect lists", "an EMA Important Medical Event", and "additionally life-threatening"; $A(e)$ indicates that a pair documenting e also shares a protein target with a third drug (the Tier 2 trigger); and $I(e)$ is an inverse pair-frequency term,

$$I(e) = \log\,(\ N\ /\ \max(n_e,\ 1)\ ), \qquad (4.2)$$

with N the number of distinct documented pairs and n_e the number documenting e, so effects specific to few pairs outweigh ubiquitous ones. The weights are $w_r = 1.0$, $w_i = 0.5$, $w_m = 1.0$, $w_s = 2.0$, $w_c = 2.0$, $w_a = 1.0$, set by hand and exposed as configuration for ablation. In Decagon, $M(e)$ equals 1 for every effect (4,020 of 4,020 effects in 40 random regimens), because polypharmacy effects are by definition not attributed to single drugs; it therefore adds a constant and does not change the order.

4.1.4 Large Language Model Usage

The LLM is reached through OpenRouter's OpenAI-compatible `/chat/completions` endpoint, so the model can be swapped through `LLM_MODEL` without code changes; the default is Google's `gemini-2.5-flash`. It is used as a constrained explanation layer over evidence that was already retrieved, ranked and bounded. Each request asks for `json_object` output with the model's extended reasoning disabled, to keep latency and cost low, and uses `max_tokens` = 4000, a 120 s timeout (15 s to connect) and up to three tries with back-off on connection failure only. The content of each prediction is fixed by the serialised evidence and checked afterwards.

The system prompt fixes the output contract: a JSON object with a `predictions` array whose items carry `side_effect`, `tier`, one or more `evidence_ids`, and a Vietnamese `explanation` of one or two short sentences describing the side effect in plain words (the interface lists the drugs). It restates the tier semantics of Table 3.2 as hard constraints: Tier 1 must reproduce a documented pairwise effect; Tier 2 additionally requires a target shared by at least three drugs; Tier 3 is a hypothesis, allowed only when a retrieved bridge links an undocumented pair, with its label copied verbatim from the bridge's candidates. Output is capped at 15

predictions (Tier 1 first, at most two Tier 3), and the model must return an empty array rather than invent evidence.

4.1.5 Grounding, Validation, and Fallback Behaviour

The response is parsed against a strict Pydantic schema; if the model wrapped the JSON in a code fence or stopped mid-stream, a tolerant path keeps every complete, schema-valid object found before the cut-off instead of discarding the response. Parsing success does not make a prediction trustworthy, so each item is re-validated against the retrieved subgraph: every cited evidence identifier must exist and carry the declared tier; for Tiers 1 and 2 the label must match every cited item, and for Tier 3 it must be among the bridge's candidate labels; each cited edge is checked against the target, PPI and side-effect tables; duplicates are removed. A Tier 2 item that fails is downgraded to Tier 1 when the evidence it cited already includes documented Tier 1 items for the same label; nothing new is added at this step. Any other failing item is dropped and logged with its reason (unknown identifier, tier mismatch, invalid path, label mismatch), so the rate of fabricated evidence can be audited. This check covers evidence and labels, not the wording of the explanation.

If the LLM call fails (no API key, non-conforming JSON, HTTP or network error), the pipeline still returns the ranked evidence and coverage summary, with template explanations and a diagnostic message.

4.2 Evaluation

4.2.1 What can and cannot be measured

Decagon records only whether a side effect was reported for a pair of drugs. It has no ground truth for regimens of three or more drugs, so no accuracy figure for a whole regimen can be computed. The evaluation therefore asks three narrower questions: whether every result can be traced to the source data (Section 4.2.2); whether well-established dangerous interactions stay visible as the regimen grows (4.2.3); and how far the Tier 3 hypotheses are supported (4.2.4). All numbers come from scripts in the repository, with fixed seeds and without calling the LLM.

4.2.2 Faithfulness to the data

For 200 random regimens of 3–6 drugs, every returned effect and evidence path was checked against the original CSV files, read independently of the SQLite cache (Table 4.4). The output matches the data exactly; this shows that the system reproduces what is recorded, not that the recorded effects occur in a patient taking the regimen. The output check of Section 4.1.5 was tested on 1,250 predictions, 300 valid and 950 deliberately corrupted (invented evidence identifiers, wrong tiers, wrong labels): all valid ones were kept and all corrupted ones rejected. The name-recognition tests use inputs written by the authors, so they do not show performance on arbitrary users.

4.2.3 Known interactions in larger regimens

The benchmark has 57 well-known pairwise interactions (for example warfarin with an NSAID, the NSAID, ACE inhibitor and diuretic "triple whammy", an opioid with a benzodiazepine), each with its expected clinical effect, chosen from standard pharmacological knowledge and not reviewed by a clinician; two contain a drug absent from Decagon and are skipped, leaving 55.

Table 4.4: Faithfulness and robustness checks

Check	Passed / total	Rate
Tier 1 effects returned are in the combo CSV (precision)	52,880 / 52,880	100%
Tier 1 effects of the returned pairs are all present (recall)	52,880 / 52,880	100%
Tier 2 pairs satisfy the shared-target rule	72 / 72	100%
Evidence paths correct edge by edge (Tiers 1–3)	40,529 / 40,529	100%
Corrupted LLM predictions rejected	950 / 950	100%
Valid LLM predictions kept	300 / 300	100%
Drug names recognised, 71 typed inputs (none wrong, 1 asked back)	70 / 71	98.6%
Chat sentences with the right drugs, 30 sentences	30 / 30	100%

Each is a three-drug regimen with one interacting pair and a bystander. A hit means the expected effect is among the first 3 or 10 entries of the serious-effect list (the interface shows 10). To test larger regimens, each case was extended with 1, 2, 3 or 5 random bystander drugs from the other cases (five draws per case and size). Separately, 20 regimens of 4–8 drugs were written by hand, each with one known interacting pair; for two, Decagon has no record of the expected effect for the pair, so they cannot be scored and are left out (18 remain). Table 4.5 gives the results; chance is the probability that a random ordering of the serious effects puts the expected one in the first 10.

Table 4.5: Known interaction recovered, by regimen size

Regimens	Runs	Hit@3	Hit@10	Chance@10	MRR
3 drugs (benchmark)	55	23.6%	52.7%	11.7%	0.208
4 drugs (+1 bystander)	275	18.2%	47.6%	8.4%	0.189
5 drugs (+2)	275	12.7%	44.4%	6.7%	0.149
6 drugs (+3)	275	14.5%	43.3%	5.8%	0.153
8 drugs (+5)	275	11.6%	40.0%	4.9%	0.150
4–8 drugs, hand-written	18	11.1%	27.8%	6.4%	0.134

The signal weakens as drugs are added, from 52.7% at three drugs to 40.0% at eight (hit@3: 23.6% to 11.6%), and is lower still in the hand-written regimens (5 of 18 in the top 10), but stays several times above chance. Two reasons explain the decline: the data record presence only, so one specific interaction competes with hundreds of other documented effects, and each added drug brings its own pairs. With 18 cases, one or two cases move a rate by 5 to 10 points, so the rows show a common trend, not precise estimates. In a regimen of four or more drugs the interface, which shows 10 serious effects, will often not display the effect of a known dangerous pair.

4.2.4 Tier 3 hypotheses

Tier 3 rests on protein bridges. Used alone to decide whether a pair of drugs has any documented interaction, protein links between targets give an AUROC of 0.54, close to chance (0.50), while the popularity of the drugs alone gives 0.94, so bridges are not evidence that a pair interacts. For 2,000 pairs with their data hidden, the label suggested for Tier 3 has precision@10 of 0.507, against 0.418 for the globally most common effects and 0.065 for random labels; the per-effect ranking test shows the same pattern (AUROC 0.951 against 0.908 for popularity). Much of what

the data support reflects how often a drug is reported, not a mechanism, so Tier 3 is a hypothesis for review, never a finding.

4.3 Limitations

Data. Only the 645 Decagon drugs are covered (others are reported as absent, not guessed), and only 284 have known targets, so the other 361 yield Tier 1 only. Only 30.6% of the 207,690 possible pairs have a recorded effect, and a missing pair is not a safe pair. The data hold no dose, frequency, severity, age or comorbidity, and come from spontaneous reports with reporting bias and noise.

Three or more drugs. All evidence is pairwise: a regimen of k drugs is treated as its $k(k-1)/2$ pairs, ranked and filtered. The system screens for dangerous pairs inside a regimen; it does not identify effects that arise only when three or more drugs are taken together. Tier 2 marks a shared target, a structural clue not shown to amplify an effect: the target table records no direction of action (agonist, antagonist, inhibitor), and two drugs at one receptor can as well oppose each other. Widening Tier 2 to proteins within one PPI hop of three drugs' targets (an option, off by default) moved hit@10 by under two points either way on the benchmarks of Table 4.5, within the noise of so few cases, and setting the Tier 2 ranking weight to zero left the three-drug hit@10 unchanged. Metabolic interactions (CYP enzymes, transporters, QT prolongation), which cause many dangerous combinations, are not represented as such.

Accuracy. The strongest results (Table 4.4) measure agreement with the data, not clinical correctness. The clinical tests are small, chosen by the authors and list one expected effect per case: about half of the known interactions reach the top 10 in three-drug regimens, but 5 of 18 (27.8%) in hand-written regimens of four to eight drugs, so the displayed list often misses a known dangerous pair in larger regimens. The ranking weights were not tuned on these cases.

Input and language model. Drug names are recognised by rules, not by a learned model: drug classes ("anti-inflammatory"), negation ("allergic to aspirin") and unusual typos are not handled, and unrelated words are sometimes proposed as drugs for the user to reject. The language model only writes a short general description of each side effect, which comes from its general knowledge rather than from the data, was not evaluated, and is labelled as AI-written in the interface. The output is population-level information, not medical advice.

Conclusion and Future Work

This report presented a Graph-RAG pipeline that takes a regimen of three or more drugs, retrieves the pairwise side-effect history and protein evidence for it from the Decagon graph, ranks the effects with a transparent score, and asks a single API-based LLM to describe them, with every cited item re-checked against the data.

The results support a modest conclusion. The pipeline reproduces the data exactly and rejects unsupported model output (Table 4.4), and it ranks known dangerous pairs well above chance, though less so as regimens grow: 52.7% in the top 10 for three drugs and 27.8% for hand-written regimens of four to eight (Table 4.5). The protein-based Tier 2 and Tier 3 evidence adds little beyond how often a drug is reported. The system is therefore a screening and explanation aid over recorded pairwise reports, not a predictor of new interactions.

Future work follows from these limits: review of the benchmark by a clinician and a larger set of high-order regimens, for example from HODDI [8]; frequency information from the underlying spontaneous reports, to replace presence-only ranking with a proper disproportionality score; a trained graph model for calibrated probabilities on untested combinations (Section 1.2.2); a per-pair view for large regimens, so that the top-10 cut-off cannot hide a known dangerous pair; and recognition of drug classes and negation in the input.

Bibliography

[1] M. Zitnik, M. Agrawal, and J. Leskovec, "Modeling polypharmacy side effects with graph convolutional networks (Decagon)," *Bioinformatics*, vol. 34, no. 13, pp. i457–i466, 2018.

[2] H. Xu, S. Sang, and H. Lu, "Tri-graph information propagation for polypharmacy side effect prediction (TIP)," *arXiv:2001.10516*, 2020.

[3] A. K. Nyamabo, H. Yu, and J.-Y. Shi, "SSI-DDI: substructure–substructure interactions for drug–drug interaction prediction," *Briefings in Bioinformatics*, vol. 22, no. 6, bbab133, 2021.

[4] Y. Wang, Z. Yang, and Q. Yao, "Accurate and interpretable drug–drug interaction prediction enabled by knowledge subgraph learning (KnowDDI)," *Communications Medicine*, 2024.

[5] D. Wu, W. Sun, Y. He, Z. Chen, and X. Luo, "MKG-FENN: A multimodal knowledge graph fused end-to-end neural network for DDI prediction," *AAAI*, vol. 38, no. 9, pp. 10216–10224, 2024.

[6] C. Xu, K. C. Bulusu, H. Pan, and O. Elemento, "DDI-GPT: Explainable prediction of drug–drug interactions using LLMs enhanced with knowledge graphs," *bioRxiv*, 2024.

[7] G. De Vito, F. Ferrucci, and A. Angelakis, "LLMs for drug–drug interaction prediction: A comprehensive comparison," *arXiv:2502.06890*, 2025.

[8] Z. Wang et al., "HODDI: A dataset of high-order drug–drug interactions for computational pharmacovigilance," *arXiv:2502.06274*, 2025.

[9] Z. Shen, Y. Liu, Q. Yao, et al., "DDIAgents: Mechanism-conditioned context flow for drug–drug interaction prediction," *arXiv:2606.31085*, 2026.

[10] Z. Liu et al., "DrugReason: Dynamic multi-view reasoning over knowledge graph and language evidence," *arXiv:2609.06779*, 2026.